FTML practical session 7

17 mai 2026

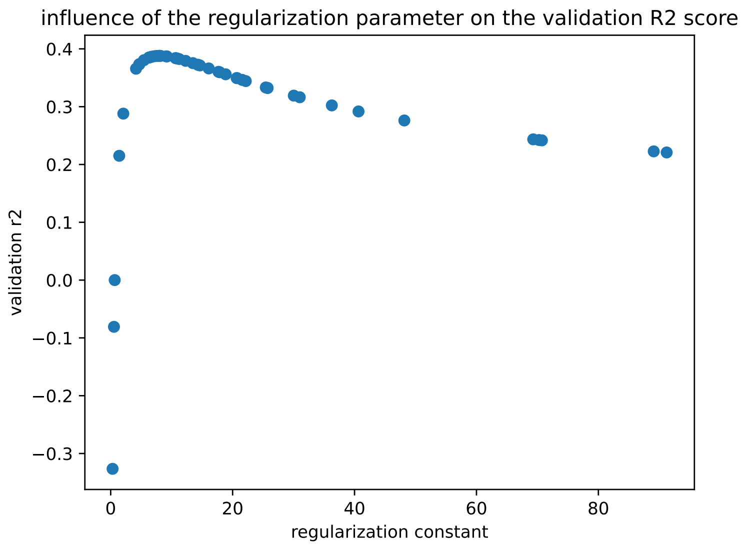


TABLE DES MATIÈRES

1 HYPERPARAMETER TUNING METHODS

In this exercise we summarize the most common hyperparameter tuning methods and use **optuna** as a bayesian optimization framework.

1.1 Grid search and random search

Grid search is the most basic approach to hyperparameter tuning : choose a list of values for each and iterate through a grid containing all possible combinations.
https://scikit-learn.org/stable/modules/grid_search.html#

See also :

— random search

— successive halving.

1.1.1 *Issues*

Grid search might be suboptimal in the sense that all hyperparameter combinations will be tried, although many of them might be irrelevant. Also, the size of the grid is exponential in the number of parameters.

1.1.2 *Question*

Why can we often not run a gradient-based optimization algorithm to look for better HP values ?

1.2 Bayesian optimization

More modern approaches use smarter optimization methods to tune the hyperparameters. For instance, **Bayesian optimization**.

https://en.wikipedia.org/wiki/Bayesian_optimization

1.2.1 *Frameworks*

Several frameworks exist to perform Bayesian optimization over HPs values, and automate the search.

skopt https://scikit-optimize.github.io/stable/

optuna https://optuna.org/

With these methods, we can specify ranges for scalar HP values, instead of grids of values.

1.2.2 *Optuna and optuna dashboard*

Optuna creates "studies", in which HP sets are tested in a sequence. A set of HPs is tested in a "trial".

https://optuna.readthedocs.io/en/stable/reference/study.html

Some interesting optuna aspects to consider :

— some trials can be "pruned" : they are considered as not interesting by optuna, and are discarded before the optimization is finished.

— it is possible to start a study, stop the program, and resume the study later, with the previous trials still in the study database.

— you can analyze the results of a study by processing the study object with other libraries like pandas, seaborn, matplotlib, etc.

With optuna, you can display and analyze the influence of the HP with a dashboard that opens in your browser.
`https://github.com/optuna/optuna-dashboard`

Perform HP optimization with optuna over a problem of your choice. Visualize the results in the optuna dashboard

Suggestion : if you want, you can use **main_optuna_template.py** and the data contained in **data/**, in **exercise_3_optuna/** folder (for apply a ridge regression on this dataset), but you are encouraged to try also in a different problem, for instance with scikit's toy datasets or any dataset of your choice.

2 INFLUENCE ON DATA SCALING ON THE CONVERGENCE OF SGD

We would like to perform a binary clasification on the following 3 datasets (figures 1, 2, 3). Each one has a different difficulty for a linear classifier, such as a SVM. We also note that the scales are different on the x and y axes.

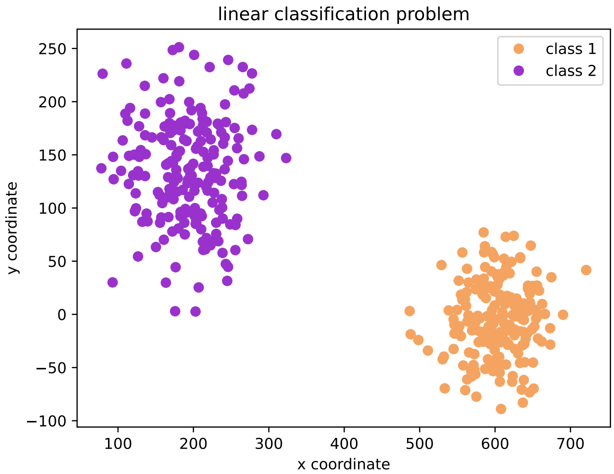


Figure 1 – Dataset 1

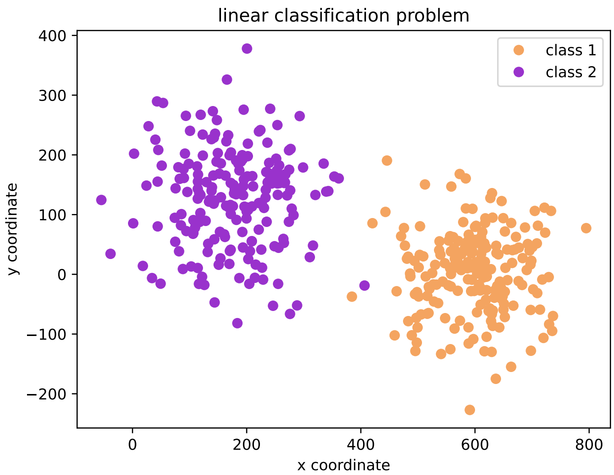


Figure 2 – Dataset 2

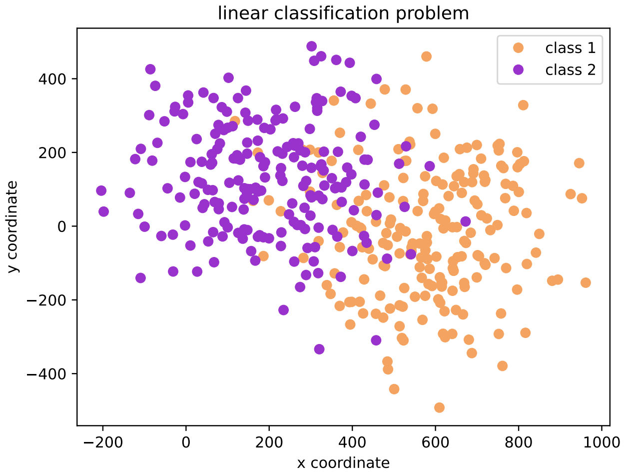


FIGURE 3 – Dataset 3

The data are located in **exercice _3 _svm_sgd/data/**.

Data standardization consists in transforming the data so that each feature (each column) is centered (zero mean) and has a variance equal to 1. It is experimentally often noticed that algorithms trained by SGD give a better performance (generalization error) when the data are standardized.

https://scikit-learn.org/stable/modules/preprocessing.html#preprocessing-scaler.

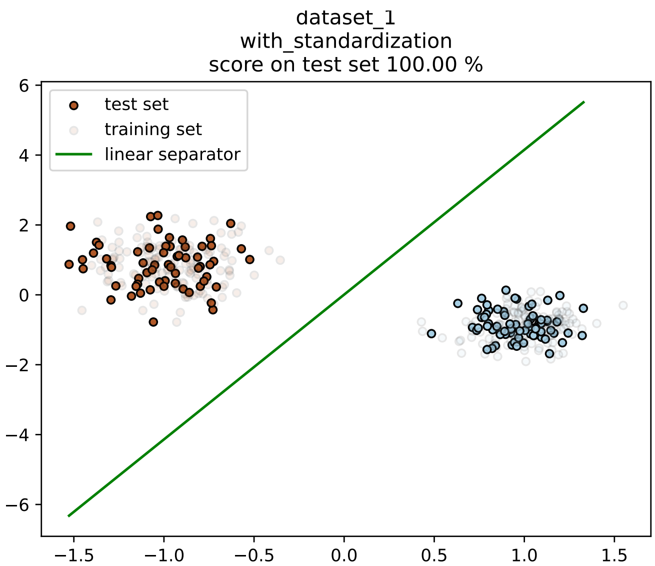
Perform a linear classification on these data, optimized by SGD, and compare quality of the results with and without preprocessing the data by standardizing them. You may use scikit-lean to do it, in particular :

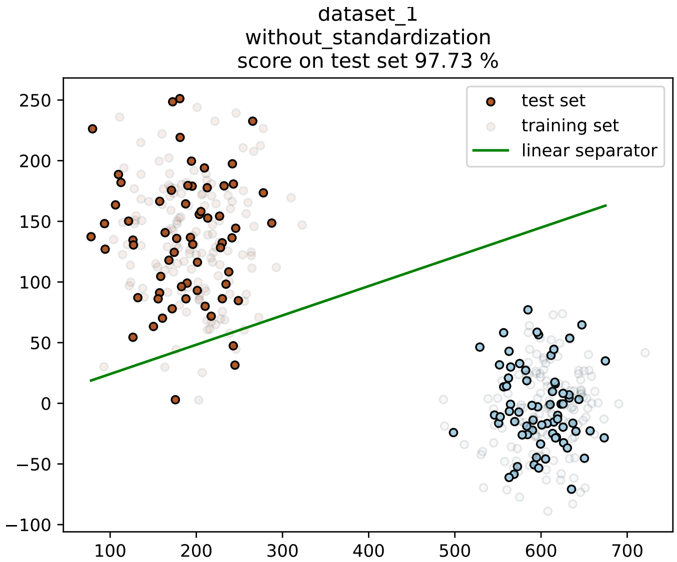
— https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.SGDClassifier.html. By default, this class trains a linear SVM (no kernel).

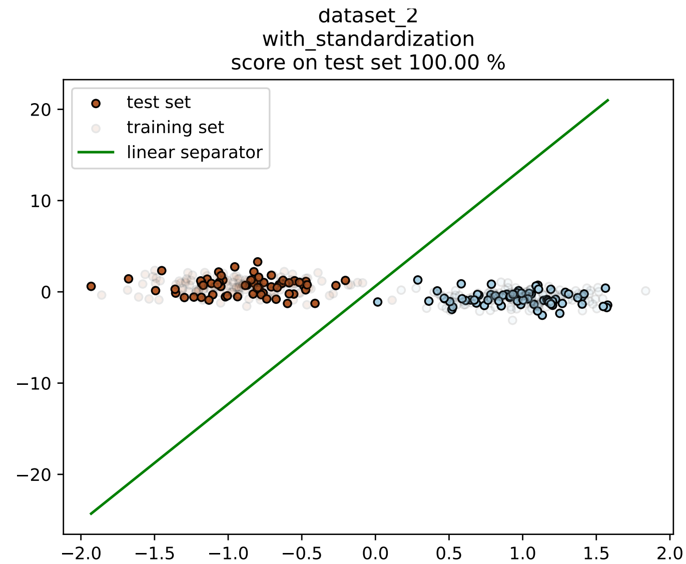
— https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html

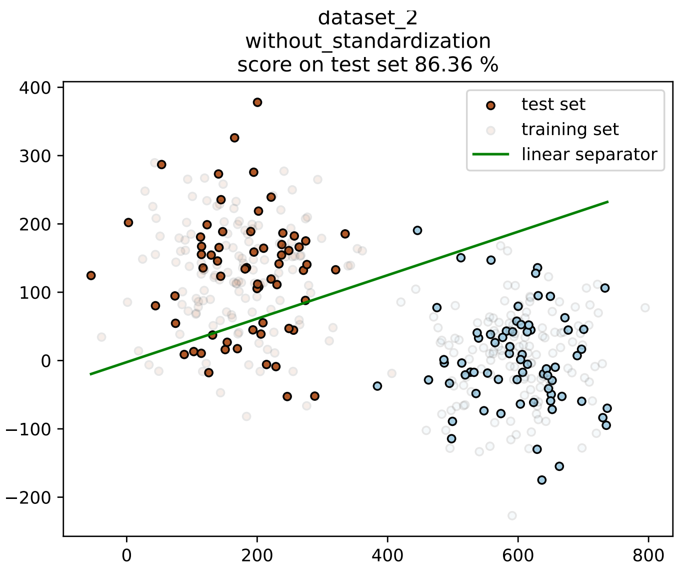
— https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html

You should obtain results like the following figures.

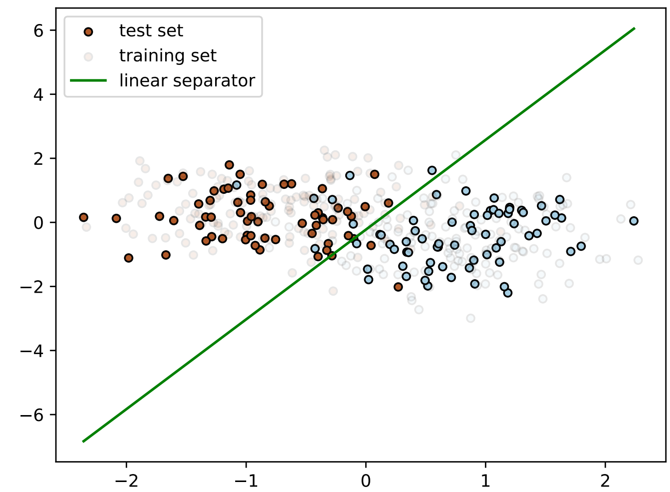



dataset_1
without_standardization
score on test set 97.73 %
test set
training set
linear separator
250
200
150
100
50
0
−50
−100
100
200
300
400
500
600
700

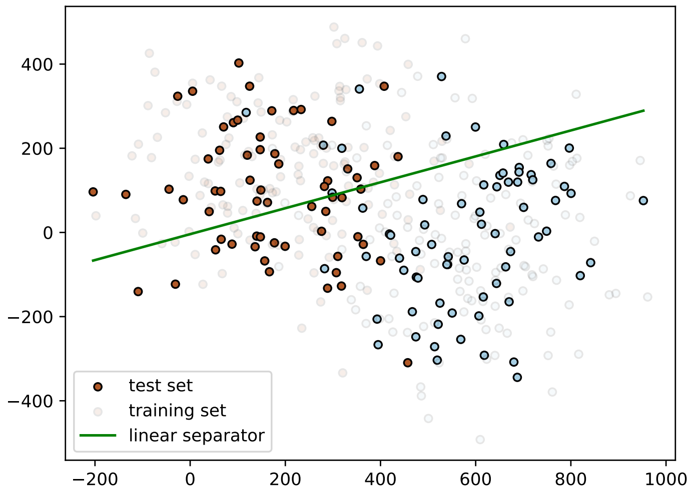

dataset_2
with_standardization
score on test set 100.00 %
test set
training set
linear separator
20
10
0
−10
−20
−2.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0


dataset_2
without_standardization
score on test set 86.36 %
test set
training set
linear separator
400
300
200
100
0
−100
−200
0
200
400
600
800

dataset_3
with_standardization
score on test set 90.15 %



dataset_3
without_standardization
score on test set 72.73 %



In this example, we saw that data standardization may give an improved performance, on a linear SVM trained by SGD. You can perform the same study on different datasets such as the toy datasets from scikit or other datasets.